

```
IN  grep -rn --include="*.c" --include="*.cpp" --include="*.h" --include="*.hpp" -l "dcms_mcu_api" /home/c-ziming.jiang/dsar_app_fw_platform_vip 2>/dev/null

OUT  <persisted-output>
      Output too large (45.5KB). Full output saved to: /home/c-ziming.jiang/.claude/projects/-home-c-ziming-jiang-dsar-app-fw-platform-vip/aed1878b-7e07-445a-94e2-862e335ab33c/tool-results/bg9ao244x.t
```

- Read minidcos_if.cpp
- Read dji_bf_app.cpp
- Read param_protocol_client.c
- Bash Find handle_msg in dcms_adp_dcos.cpp

```
IN  grep -rn --include="*.cpp" "DcmsAdpNode::handle_msg" /home/c-ziming.jiang/dsar_app_fw_platform_vip 2>/dev/null

OUT  /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq-plat/dsar-plat-bf/dsar_app/cdd/dcms_adapt/dcms_adp_dcos.cpp:395:         DcmsAdpNode::handle_msg(topic, origin_msg);
      /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq-plat/dsar-plat-bf/dsar_app/cdd/dcms_adapt/dcms_adp_dcos_ext.cpp:27:int32_t DcmsAdpNode::handle_msg(DcmsAdpTopic &topic, const dcoss::ConstMsg
```

- Bash Find dcms_adp_internal.h

```
IN  grep -rn --include="*.h" -l "dcms_adp_internal" /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq-plat/dsar-plat-bf/ 2>/dev/null | head -5

OUT  (Bash completed with no output)
```

- Read cdd_config.h
- Read init.h
- Thought for 8s >
- Let me read the key entry point files to show the exact initialization difference.

- Read dji_uds_service_v4.cpp
- Thought for 2s >
- Read dji_ad_app.cpp
- Thought for 3s >
- Read dcms_mcu_api.cpp (lines 441-600)

- Thought for 2s >

- Bash Find dcms_mcu_api.cpp location

```
IN  find /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq-plat -name "dcms_mcu_api.cpp" -type f 2>/dev/null | head -5

OUT  /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq-plat/dsar-plat-bf/dsar_app/cdd/dcms_adapt/dcms_mcu_api.cpp
```

- Thought for 4s >

- Bash Find dcms_mcu_api.cpp in workspace

```
IN  find /home/c-ziming.jiang/dsar_app_fw_platform_vip -path "**/dcms_adapt/dcms_mcu_api.cpp" -type f 2>/dev/null

OUT  /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq-plat/dsar-plat-bf/dsar_app/cdd/dcms_adapt/dcms_mcu_api.cpp
```

- Thought for 3s >

- Bash Check dsar-hq-plat submodule contents

```
IN  ls /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq-plat/

OUT  dsar-plat-ad
      dsar-plat-bf
      dsar-sip
```

- Thought for 7s >

- Read dcms_mcu_api.cpp (lines 441-600)

- Thought for 2s >

- Read dcms_adp_minidcos.cpp (lines 2-101)

- Thought for 2s >

- Read dcms_adp_dcos.cpp (lines 2-101)

- Thought for 9s >

- 现在有足够的信息了。以下是完整的分析

MINIDCOS_DOMAIN vs DCOS_DOMAIN 详解

一、两个域的本质区别

DCOS_DOMAIN (0)	MINIDCOS_DOMAIN (1)
底层: DDS (Data Distribution Service) 中间件	底层: 共享内存 Shared Memory (20MB 内存池)

```
需要: dcos::Init() 全局初始化      需要: minidcos_if_create_instance()
      dcos::NodeHandle 创建节点      + minidcos_if_reg_module3()

特点: 完整的发布/订阅/服务框架      特点: 轻量级, 去掉了 DDS 依赖
      支持 QoS 策略                    仅保留核心通信能力
      支持跨网络发现                  仅支持同芯片跨核 IPC

用途: 主 SOC 应用之间通信            用途: DoIP 诊断进程与 MCU 通信
      (dji_ad_app, dji_hr_app)        (dji_doip_service_ipv4/v6)
```

为什么 DoIP 不能用 DCOS? ——DoIP 是独立进程 (通过 `dlopen` 加载 `.so` 运行动态库), 如果启动完整的 DDS 中间件会有两个问题: 一是 DDS 初始化慢 (需要网络发现、节点注册), 影响诊断仪连接响应时间; 二是 DoIP 进程生命周期独立, DDS 的节点退出/发现机制会引入不必要的复杂性。所以用 MiniDCOS —— 一个纯共享内存的轻量级 IPC, 直接通过共享内存与 MCU 通信。

二、域的设置 —— 进程级别决定

每个 SOC 进程在 `main()` 的第一行代码就决定了自己的域:

DCOS 进程 (`dji_ad_app.cpp:69`):

```
config.CddConfig().process_name = "dji_ad_app";
config.CddConfig().dcos_domain = DCOS_DOMAIN; // - 设为 DCOS
dsar_plat_init(config);
```

```
dcos::Init(process_name); // - 启动 DDS 中间件
```

MiniDCOS 进程 (`dji_uds_service_v4.cpp:40-44`):

```
config.CddConfig().process_name = "dji_doip_service_ipv4";
config.CddConfig().dcos_domain = MINIDCOS_DOMAIN; // - 设为 MiniDCOS
dsar_plat_init(config);
```

```
// 注意: 没有 dcos::Init() !
// 而是由上层 appl_ipv4_main() 调用 uds_minidcos_service_init() 创建共享内存实例
```

这个 `dcos_domain` 值被写入全局变量 `g_cdd_config.dcos_domain` (`cdd_config_impl.cpp`), 之后所有 DCMS API 调用都读取它来决定走哪条路径。

三、域的路由 —— 每次发送都判断

在 `dcms_mcu_api.cpp:478-491`, 每个 `send` 函数都有完全相同的 if-else 分支:

```
int32_t dcms_mcu_topic_send_timeout(uint32_t topic_id, const uint8_t *data,
                                     uint32_t data_len, uint32_t timeout_us)
{
    DcmsAdpTopic *dcms_mcu_topic = get_dcms_mcu_topic_by_topic_id(topic_id);
    // ... 参数校验 ...

    if (DCOS_DOMAIN == dsar_plat_bf::cdd::g_cdd_config.dcos_domain)
    {
        // === DCOS 路径: 调 DDS Publisher/ServiceClient ===
        ret = DcmsAdpNode::send_dcos_topic_raw(dcms_mcu_topic, data, data_len,
                                                NULL, NULL, g_sensor_time_us,
                                                sys_time, timeout_us);
    }
    else if (MINIDCOS_DOMAIN == dsar_plat_bf::cdd::g_cdd_config.dcos_domain)
    {
        // === MiniDCOS 路径: 调共享内存 publish/call_service ===
        ret = DcmsAdpNode::send_minidcos_topic(dcms_mcu_topic, data, data_len,
                                                nullptr, nullptr, g_sensor_time_us,
                                                get_system_time_us_64bits(), timeout_us);
    }
    return ret;
}
```

所有 6 个 `send` 变体 (`topic_send`、`send_timeout`、`send_sync`、`send_with_sensor_time`、`send_with_sensor_time_with_sys_time`、`send_ext`) 都遵循这个模式。

四、两条底层路径的具体实现

DCOS 路径 — 走 DDS 中间件

`dcms_adp_dcos.cpp:132-236`:

```
int32_t DcmsAdpNode::send_dcos_topic_raw(DcmsAdpTopic *topic_info_ptr, ...)
{
    if (topic_info_ptr->topic_cfg->is_service_client)
    {
        // Client 模式: 调 dcos::ServiceClient->Call()
        dcos::Msg<dcos::Any> req_msg;
        // ... 设置 sensor_time, sys_time ...
        if (timeout_us != 0)
            topic_info_ptr->client->Call(&req_msg, &ack_msg, timeout_us); // 同步
        else
            topic_info_ptr->client->Call(&req_msg); // 异步
    }
    else // Publisher 模式
    {
        // 调 dcos::Publisher->Publish()
        topic_info_ptr->publisher->Publish(&msg_anys);
    }
}
```

DCOS 内部走的是 DDS 协议栈: 序列化 → UDP 多播发现 → 网络传输 (或本地 IPC 优化)。

MiniDCOS 路径 — 走共享内存

dcms_adp_minidcos.cpp:32-157:

```
int32_t DcmsAdpNode::send_minidcos_topic(DcmsAdpTopic *topic_info_ptr, ...)
{
    if (topic_info_ptr->topic_cfg->is_service_client)
    {
        // 分配共享内存消息
        const auto msg = minidcos_if_msg_new(data_len);
        // 拷贝数据到共享内存
        memcpy(minidcos_if_msg_get_entity(msg), data, data_len);

        if (timeout_us != 0)
            // 同步：阻塞等待对端响应（共享内存 + 信号量等待）
            ret = minidcos_if_call_service_sync(topic_info_ptr->minidcos_client,
                                                msg, &resp_msg, timeout_us);
        else
            // 异步：写共享内存，通知对端，立即返回
            ret = minidcos_if_call_service(topic_info_ptr->minidcos_client, msg);
    }
    else if (topic_info_ptr->topic_cfg->publish_en)
    {
        const auto msg = minidcos_if_msg_new(data_len);
        memcpy(minidcos_if_msg_get_entity(msg), data, data_len);
        // fire-and-forget 发布
        ret = minidcos_if_publish2(topic_info_ptr->minidcos_publisher, msg);
        minidcos_if_msg_recycle(msg);
    }
}
```

MiniDCOS 的核心：一块 20MB 共享内存，读写都在这块内存上进行，通过信号量/事件通知机制唤醒对端。

五、Node 初始化时的域分支

不仅是发送，Node 的创建也按域分支（dcms_adp_dcos.cpp:45-77）:

```
DcmsAdpNode::DcmsAdpNode(const std::string& name, dcms_adp_period_t period)
{
    if (dsar_plat_bf::cdd::g_cdd_config.dcos_domain == MINIDCOS_DOMAIN)
    {
        // MiniDCOS: 注册一个 module，拿到 module_id
        const int32_t ret = minidcos_if_reg_module3(
            std::string(node_name).append("_m").c_str(), &mid);
    }
    else
    {
        // DCOS: 创建完整 DDS NodeHandle
        nh = std::make_unique<dcos::NodeHandle>(node_name);
        nh->InitNode(node_name);
    }
}
```

	DCOS 进程	MiniDCOS 进程
Node 类型	dcos::NodeHandle（DDS 节点）	uint8_t mid（MiniDCOS module ID）
注册 API	dcos::Publisher::Advertise()	minidcos_if_advertise2()
发送 API	dcos::Publisher::Publish()	minidcos_if_publish2()
接收机制	DDS Subscriber 回调	共享内存 + 事件通知回调

六、Topic 的双重身份

每个 DcmsAdpTopic 对象同时持有两套指针（dcms_adp_topic.h）:

```
class DcmsAdpTopic {
    // === DCOS 的 DDS 句柄 ===
    std::unique_ptr<dcos::Publisher> publisher; // 发布者
    std::unique_ptr<dcos::Subscriber> subscriber; // 订阅者
    std::unique_ptr<dcos::ServiceClient> client; // 服务客户端
    std::unique_ptr<dcos::ServiceServer> server; // 服务端

    // === MiniDCOS 的共享内存句柄 ===
    dcos_service_client_t* minidcos_client; // 服务客户端句柄
    dcos_service_server_t* minidcos_server; // 服务端句柄
    dcos_publisher_t* minidcos_publisher; // 发布者句柄
    dcos_subscriber_t* minidcos_subscriber; // 订阅者句柄
};
```

根据进程的 g_cdd_config.dcos_domain，注册时只填充其中一套指针，另一套保持 NULL。发送时 send_dcos_topic 用 publisher/client，send_minidcos_topic 用 minidcos_publisher/minidcos_client。

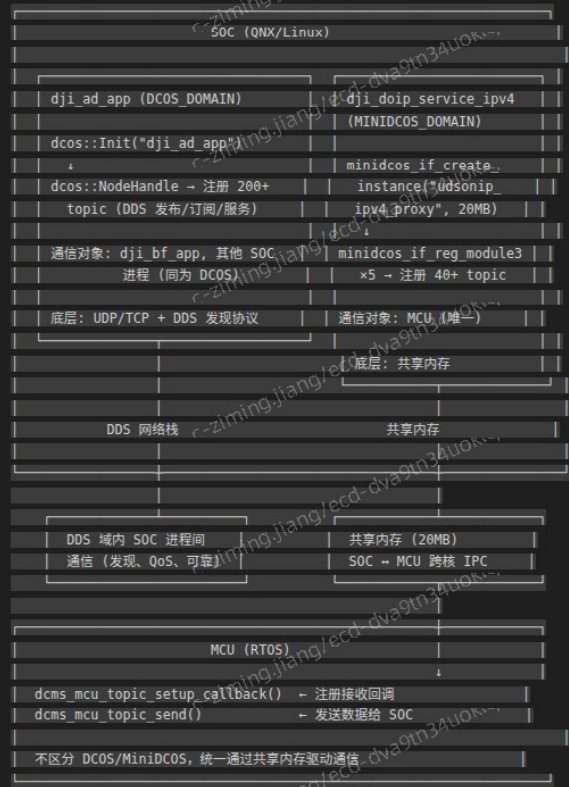
七、MCU 侧不受域影响

MCU（RTOS）侧不区分 DCOS/MiniDCOS，MCU 上运行的不是 DDS 也不是 MiniDCOS 用户态库，而是同一个共享内存驱动。所有 SOC 进程——无论 DCOS 还是 MiniDCOS——最终都是通过同一块共享内存与 MCU 通信。



图 通信框架全景图

八、通信拓扑全景图



九、如何区分一个 topic 走哪个域

有两种方式：

方式1 — 进程级默认（最常见）：进程启动时设置 `g_cdd_config.dcos_domain`，该进程注册的所有 topic 默认都走这个域。

方式2 — topic 级覆盖：`dcms_mcu_topic_cfg_t` 结构体有一个 `domain` 字段可以覆盖进程默认值。例如 `dmm_dcms_configs.h` 中绝大多数 topic 走 DCOS，但有一个明确指定：

```
// 这个 topic 显式指定走 MINIDCOS_DOMAIN，即使进程本身是 DCOS
{0, DCMS_TOPIC_GET_SOFTWARE_VERSION,
"/sys/update/app_install/firmware_version_query",
0u, 0u, 1u, 0u, MINIDCOS_DOMAIN}, // domain 字段覆盖
```

简单判断规则：

- 看进程入口：`config.CddConfig().dcos_domain` 是 `DCOS_DOMAIN` 还是 `MINIDCOS_DOMAIN`
- DoIP 诊断相关 (`dji_doip_service_ipv4/v6`) → `MINIDCOS_DOMAIN`
- 主 SOC 应用 (`dji_ad_app`、`dji_bf_app`、`log_export`) → `DCOS_DOMAIN`
- MCU 侧 → 不关心域，统一走共享内存

十、总结

维度	DCOS_DOMAIN	MINIDCOS_DOMAIN
底层技术	DDS 中间件	共享内存
需要初始化	<code>dcos::Init()</code>	<code>minidcos_if_create_instance(20MB)</code>
节点概念	<code>dcos::NodeHandle</code>	<code>minidcos_if_reg_module3()</code> → <code>mid</code>
适用进程	<code>dji_ad_app</code> 、 <code>dji_bf_app</code> 、 <code>log_export</code>	<code>dji_doip_service_ipv4/v6</code>
通信对象	SOC 进程间（一个 SOC 内多个进程）	SOC ↔ MCU（跨核）
性能	经过 DDS 序列化/网络栈	直接共享内存读写
启动速度	慢（需发现、建节点）	快（直接创建共享内存）
Topic 数	200+（AD 进程）+ 100+（BF 进程）	40+（DoIP 诊断）

一句话总结：DCOS 是 SOC 内部进程间的大巴，MiniDCOS 是 SOC 到 MCU 的专线直通车。两者共享同一套 `dcms_mcu_topic_send()` API 面，底层根据 `g_cdd_config.dcos_domain` 这个全局开关自动分流。